

Improving GIJS Key Recovery for Classic McEliece

Stephen A. Weis*
Anthropic
sweis@anthropic.com

September 2026

Abstract

Ghoshal, Ishai, Jain and Sun [13] recently gave the first distinguisher for Classic McEliece public keys that is cheaper than generic decoding, at an estimated 2^{114} to 2^{124} bit operations for the five NIST candidate parameter sets, and have since extended it to a key-recovery algorithm. The distinguisher is one large sparse linear-algebra computation. We show that this computation already contains the secret key, and we give two ways to extract it. The first takes the gradients of the polynomials that the distinguisher computes. We prove which subcode of the public code they span at each column, and the key follows from these subcodes by linear algebra, at the cost of 100 to 1400 runs of the distinguisher. The second builds on the key recovery of [13] and reads the whole support from a single run.

We also lower the cost of a run by about 20 bits, using two facts about binary Goppa codes that we prove and two heuristic assumptions that we test. Key recovery then costs 2^{94} to 2^{102} bit operations in the model of [13], against 2^{151} to 2^{287} for information-set decoding. That model does not charge for memory, and about three quarters of the operations fetch a bit from a vector of 2^{48} to 2^{52} bits. Charging for memory as the Classic McEliece security guide does gives 2^{110} to 2^{128} . The cost depends on m and t but not on the length n , and it grows by about eight bits per unit of m . Raising m to 18, 26 and 34, with n well below 2^m , puts a run above 2^{143} , 2^{207} and 2^{272} at public keys of about 0.6, 2 and 7 MB.

None of this is close to practical, and several ingredients are heuristic. We state each as an explicit assumption and test it by running the attacks end to end on small keys and, where no run is needed, at full Classic McEliece size. As a demonstration we recovered the secret key of instance 253 of the TII McEliece key-recovery challenges ($m = 8$, $t = 9$, $n = 214$), a toy-sized code that had not been solved, with two sparse kernel computations of about 10^7 unknowns and 700 core-hours each.

1 Introduction

Background. Classic McEliece [1] is the McEliece cryptosystem [18] with binary Goppa codes, essentially unchanged since 1978. The public key is a generator (or parity-check) matrix of a binary Goppa code of length n and dimension $k = n - mt$ that corrects t errors. The secret key is the support $\alpha = (\alpha_1, \dots, \alpha_n)$, a sequence of distinct elements of $\mathbb{F}_{2^m}$, together with the Goppa polynomial $\Gamma \in \mathbb{F}_{2^m}[Z]$ of degree t .

Two kinds of attack are relevant. The first is *generic decoding*, or information-set decoding (ISD). It ignores the algebraic structure, and its cost defines the security levels of the parameter sets [2, 7]. The second kind is *structural*: it tries to detect or exploit the hidden Goppa structure.

*This was prepared with Claude. I blame it for any mistakes, but take credit for the good parts.

Until 2026 no structural attack applied to the Classic McEliece parameters at a cost anywhere near ISD. The most efficient one, the higher-order-vanishing distinguisher and key recovery of [15, 14], costs 2^{298} or more [13, Table 1]. A survey that includes [11, 9, 20] is in [13, Section 1.3]. Two further structural results appeared in the first half of 2026. Bardet, Lemoine and Tillich [5] gave a polynomial-time key recovery for binary Goppa codes of very high rate. It breaks the CFS signature scheme but does not apply at the rates of Classic McEliece. Briaud, Lemoine, Randriambololona and Tillich [8] gave a key recovery that they conjecture to be subexponential. By their own estimate its distinguishing step costs 2^{454} to 2^{915} for the Classic McEliece sets.

Ghoshal, Ishai, Jain and Sun (GIJS) [13] changed this. Their distinguisher views the columns of the public matrix as points on a hidden rational curve. It looks for low-degree polynomials that vanish to high order at all columns but one. On a Goppa key such polynomials automatically vanish at the remaining column too. On a random matrix they do not. The test is a single large but sparse linear-algebra problem. GIJS estimate its cost for the five parameter sets at 2^{114} to 2^{124} bit operations [13, Section 5], which is 37 to 163 bits below ISD. They also give a heuristic decryption attack at much higher cost, and a later version of their paper adds a key-recovery algorithm. Two further key-recovery proposals that build on the same distinguisher have appeared [26, 22], and Apon [4] has proved a lower bound that puts the first of them far above ISD. Section 8 relates all of these to the present work.

This paper. We show that key recovery costs no more than a small multiple of the distinguisher, and plausibly no more than the distinguisher itself. Our contributions are the following.

1. *Sharper parameters for the distinguisher* (Section 3). We prove two facts about binary Goppa codes. The first is an identity that gives the vanishing argument t extra zeros. The second is that the sum of all public columns is one more point of the curve. Together with an exact count of the nonzero entries of the matrix and two heuristic assumptions, these lower the cost of a run from 2^{114} – 2^{124} to 2^{94} – 2^{102} .
2. *Key recovery from gradients* (Section 4). Take any polynomial that the distinguisher finds and compute its gradient at a held-out column. We prove that this gradient is a codeword of an explicit subcode of the public code: the Goppa code in which that column’s support element is removed and Γ is multiplied by the corresponding linear factor. One run yields this subcode for a few columns. About 100 to 1400 runs, depending on the parameter set, yield it for every column of a shortened code. The family of subcodes determines the key by linear algebra, which we describe and which runs in minutes at full size.
3. *Key recovery from one run* (Section 5). The key recovery of [13, Section 7] rests on the observation that the *coefficients* of the interpolation relation behind the distinguisher carry the support directly, in the manner of Sidelnikov and Shestakov [25]. We identify what stands in the way of using this at the concrete parameters, namely a very large space of trivial relations, and we give a closed form for these relations. The result is an algorithm that reads the support of a shortened code from about two hundred random kernel vectors of a single run.
4. *A challenge instance* (Section 7). We ran the single-run attack on instance 253 of the TII McEliece key-recovery challenges [17]. This is a binary Goppa code with $m = 8$, $t = 9$, $n = 214$, $k = 142$, far smaller than any code proposed for use. We recovered its secret key (Appendix A). The two kernel computations involved have 9.4 million rows and 11.8 million columns each. Every intermediate quantity came out as the analysis of Section 5 predicts.

parameter set	distinguisher		gradient attack, Sec. 4		single-run attack, Sec. 5		ISD
	[13]	Sec. 3	(a)	(b)	(a)	(b)	
mceliece348864	114.2	94.1	120.8	100.8	114.3	94.3	151
mceliece460896	120.3	99.3	128.9	109.8	120.3	99.3	191
mceliece6688128	124.0	102.4	131.0	110.2	124.0	102.4	257
mceliece6960119	123.0	101.6	130.3	109.8	123.0	101.6	257
mceliece8192128	124.0	102.4	131.0	110.2	124.0	102.4	287

Table 1: $\log_2$ of the number of bit operations, in the cost model of [13, Section 5]. Columns (a) use the conditions and cost formulas of [13] unchanged and add only statements proved in this paper. Columns (b) use the parameters of Section 3, which rest on Assumptions 1 and 2. The single-run columns rest in addition on the assumptions listed in Section 10. The first column quotes [13, Table 2], which our reimplementation of their formulas reproduces. ISD figures are those quoted in [13, Table 1] from [7]. The claimed security levels of the five sets are 2^{143} , 2^{207} , 2^{272} , 2^{272} , 2^{272} [2].

Table 1 summarises the costs. All figures are bit operations in the model of [13, Section 5], which charges neither for memory nor for moving data. This matters more here than for a cipher key search. About three quarters of the counted operations are sparse matrix–vector products, and each of these operations consumes a bit fetched from a vector of 2^{48} to 2^{52} bits (Section 9). Charging for memory in the way the Classic McEliece security guide [2] does adds 16 to 26 bits to columns (b). The attacks then remain below ISD and below the claimed levels, but by less. For the single-run attack on mceliece348864 the margin below 2^{143} shrinks from 49 bits to between 25 and 33, and for the gradient attack from 42.2 bits to between 18.3 and 26.3.

What is proved and what is not. Four statements are proved: the identity and the extra point of Section 3 (Propositions 1 and 2), the gradient theorem of Section 4 (Theorem 3), and the closed form of Section 5 (Proposition 4). The remaining ingredients are dimension counts and genericity statements of the kind that is usual in algebraic cryptanalysis. We state each one as an explicit assumption. Section 6 reports how each was tested. We ran the complete attacks on small binary Goppa keys. We ran everything that does not need the expensive step at genuine mceliece348864 and mceliece8192128 size. For the single-run attack we also have the challenge computation of Section 7. Section 10 collects the assumptions and says what would happen if one failed.

We stress that 2^{94} bit operations, organised as one sequential sparse linear-algebra computation on 2^{38} unknowns, is far beyond anything that can be run. The point is the comparison with the claimed security levels and with ISD. A second point (Section 10) is that the cost is governed by m and t and does not depend on n . It grows by about eight bits per unit of m . Parameter sets with a larger m and with n well below 2^m therefore push the cost of the attack above the claimed levels in this cost model, at key sizes that grow quadratically in m .

Verification. Every statement in this paper that is not proved was checked with an implementation. It is written in SageMath [24] and uses M4RI [3] for the large $\mathbb{F}_2$ linear algebra. It contains the cost formulas of [13, Section 5], re-typed and reproducing their Table 2, both attacks end to end on small keys, and the checks of Section 6. For each check, Section 6 states the instance sizes and the quantities observed, so that it can be repeated independently.

2 Preliminaries

Notation. We follow [13]. Let $q = 2^m$. The public key is $Y = [y_1 | \cdots | y_n] \in \mathbb{F}_2^{k \times n}$, a generator matrix of $\text{Goppa}(\alpha, \Gamma)$. Here $\Gamma \in \mathbb{F}_q[Z]$ is monic and square-free of degree t (irreducible in Classic McEliece), and $k = n - mt$. Write $\Pi(Z) = \prod_j (Z - \alpha_j)$ and $D = n - 2t - 1$.

Everything rests on one fact: a binary Goppa code lies in a generalised Reed–Solomon (GRS) code of dimension $D + 1$ [13, Fact 2.3]. There are multipliers $\lambda_j = \Gamma(\alpha_j)^2 / \Pi'(\alpha_j)$ and, for every codeword c , a polynomial f_c of degree at most D with $c_j = \lambda_j f_c(\alpha_j)$ for all j . We call f_c the *code polynomial* of c . It can be written down explicitly. Let $F = \prod_{c_j=1} (Z - \alpha_j)$ and $\bar{F} = \Pi/F$. Then $\sum_j c_j / (Z - \alpha_j) = F'/F$, and $c \in \text{Goppa}(\alpha, \Gamma)$ means $\Gamma \mid F'$. In characteristic 2 the derivative F' is a square, and Γ is square-free, so in fact $\Gamma^2 \mid F'$. The code polynomial is

$$f_c = F' \bar{F} / \Gamma^2. \quad (1)$$

Applying this to the rows of Y gives $y_j = \lambda_j p(\alpha_j)$, where $p = (f_1, \dots, f_k)$ is a vector of polynomials of degree at most D . In other words, up to the scalars λ_j , the columns of Y are points of a rational curve of degree D in $\mathbb{F}_q^k$, given in unknown coordinates.

The instance. The distinguisher of [13, Sections 3–5] does not work with the full public code. It shortens the code on ℓ coordinates and brings the result to systematic form. This leaves a code with $k_\ell = k - \ell$ rows and $n_\ell = k_\ell + mt$ columns. It is again a Goppa code, with the same Γ and the surviving support elements, and we set $D_\ell = n_\ell - 2t - 1$. We call this shortened code in systematic form *the instance*. Its k_ℓ *unit columns* are unit vectors. Its other mt columns are the *non-unit columns*.

The jet matrix. Fix a degree d and a multiplicity $s < d$. Consider homogeneous multilinear polynomials h of degree d in k_ℓ variables. Such an h is a vector of $N = \binom{k_\ell}{d}$ coefficients h_A , one for each d -set A of variables. For a non-unit column y_j and an order $u < s$, requiring all Hasse derivatives of order u of h to vanish at y_j is a set of linear conditions on the coefficients. The condition of order 0 is $h(y_j) = 0$, and we call it the *value condition*. A column at which the conditions of all orders below s_j are imposed is a *constraint column* of *multiplicity* s_j . Unit columns need no conditions: a multilinear form of degree d vanishes to order $d - 1$ at a unit vector, so all conditions of order at most $d - 2$ hold there automatically.

Collect the conditions as the columns of a binary matrix E with N rows. This is the *jet matrix* of [13, Section 7]. A column of E is indexed by a pair (j, S) , where j is a constraint column and S is a set of fewer than s variables (the derivative index). The entry in row A and column (j, S) is 1 exactly when $S \subseteq A$ and y_j is 1 on $A \setminus S$. The columns of E that belong to one y_j form the *block* of y_j .

The distinguisher. The *row kernel* $\{h : hE = 0\}$ is the space of polynomials that vanish to order s at every constraint column. The distinguisher keeps one non-unit column y_τ out of the constraint set. We call it a *held-out column*. The distinguisher computes a nonzero h in the row kernel and tests whether $h(y_\tau) = 0$.

Here is why this works [13, Theorems 3.2 and 4.1]. For h in the row kernel, consider the univariate polynomial $\varphi_h(Z) = h(p(Z))$. It has degree at most dD_ℓ . By the chain rule and homogeneity it vanishes to order s at α_j for every constraint column y_j . It vanishes to order $d - 1 \geq s$ at the support elements of the unit columns. Suppose the number of these forced zeros

exceeds dD_ℓ . For $d = s + 1$ this condition reads

$$s(n_\ell - 1) > dD_\ell \quad [13, \text{condition (6)}]. \quad (2)$$

Then $\varphi_h \equiv 0$, and in particular $h(y_\tau) = \lambda_\tau^d \varphi_h(\alpha_\tau) = 0$. On a random matrix, $h(y_\tau)$ behaves like a fair coin as long as the row kernel is nontrivial. To guarantee a nontrivial row kernel, GIJS require more monomials than conditions:

$$\binom{k_\ell}{d} > mt \sum_{u < s} \binom{k_\ell}{u} \quad [13, \text{condition (7)}]. \quad (3)$$

The cost of the distinguisher is the cost of finding kernel vectors of the sparse matrix E . In [13, Section 5.3] this is done with a block Lanczos algorithm, and its cost is given by explicit formulas. We reuse these formulas without change except where stated. We write *a run* for one such kernel computation.

The column kernel. The same matrix has a *column kernel* $\ker E = \{\rho : E\rho = 0\}$. This is the space of linear relations among the derivative conditions. In [13] it is simply “the kernel of the jet matrix”. We say row kernel and column kernel to keep the two apart. The two are linked. The statement “ $h(y_\tau) = 0$ for every h in the row kernel” is equivalent to the statement “the value condition at y_τ is a linear combination of the other conditions”. The latter says that a particular column-kernel vector exists once the value condition at y_τ is added to E . Section 4 works with the row kernel, and Section 5 works with the column kernel.

Frobenius conjugates. The matrix E is binary. It therefore cannot tell the support α from its Frobenius conjugates $\alpha^{2^u} = (\alpha_1^{2^u}, \dots, \alpha_n^{2^u})$, $u = 0, \dots, m - 1$. Each conjugate is the support of an equivalent key, because $\text{Goppa}(\alpha, \Gamma) = \text{Goppa}(\alpha^2, \Gamma^{(2)})$, where $\Gamma^{(2)}$ is Γ with squared coefficients. Any structural key recovery must at some point pick one of the m conjugates and stay with it across all columns. We call the conjugates *branches*. Both attacks below contain a step that selects a branch.

The toy key. Several experiments use the same small binary Goppa code, which we call the *toy key*: $m = 6$, $t = 6$, $n = 64$, $k = 28$, not shortened.

3 Sharper parameters for the distinguisher

We reimplemented the cost analysis of [13, Section 5]. Our version reproduces its Table 2 to the two decimals given there. Four observations tighten the analysis. The first and the last are exact. The second is a standard heuristic. The third is a proved identity that we use with a mild heuristic. Together they move every parameter set from $(d, s) = (8, 7)$ to $(7, 6)$. This is where the twenty bits come from, since each unit of s costs eleven to twelve bits.

1. Sparsity (exact). GIJS bound the number of nonzero entries that one constraint column contributes to one row of E by $\sum_{u < s} \binom{d}{u}$. The true number follows from the description of E above. If a of the d variables of the monomial are 0 at y_j , the number of nonzero entries is $\sum_{u < s} \binom{d-a}{u-a}$. In systematic form the non-unit columns are balanced binary vectors, so $a \sim \text{Bin}(d, \frac{1}{2})$. The average is then 12.6 at $(7, 6)$ and 20.6 at $(8, 7)$, instead of the bounds 120 and 247. In addition, in characteristic 2 the conditions of orders $d - 2, d - 4, d - 8, \dots$ imply all the others [15, Corollary 1]. Keeping only these orders removes a few percent of the columns of E . Together this is worth about 4 bits.

2. Kernel dimension (heuristic). Condition (3) treats the conditions of different columns as independent. They are not. The standard genericity heuristic for such systems [15, Heuristic 1], [12] predicts the dimension of the row kernel as

$$\kappa(k_\ell, d, P) = [z^d] [(1+z)^{k_\ell}(1-z^2)^P]_+. \quad (4)$$

Here P is the number of points at which conditions are imposed, $[z^d]$ takes the coefficient of z^d , and $[\cdot]_+$ truncates the series at its first non-positive coefficient. We computed the row kernel for dozens of random matrices in systematic form and for Goppa keys, with $k_\ell \leq 28$ and $d \leq 5$. Its dimension was never below κ . It equalled κ or exceeded it by a small amount that shrinks as k_ℓ grows. At the parameters of [13, Table 2], κ is a quarter of all monomials, whereas (3) only guarantees 0.2%. The random-case argument of [13, Theorem 4.1] needs nothing more than a nonzero kernel. We therefore select parameters by $\kappa > 0$ instead of (3). This is worth about 5 bits.

Assumption 1. At the parameters of Table 2, the row kernel of a run has dimension (4).

3. Binary Goppa codes have t zeros to spare (proved identity, heuristic use). GJS observe experimentally [13, Appendix A, Table 5] that on binary Goppa keys the products of d code polynomials span $(d-1)t$ fewer dimensions than expected. The reason is the following identity.

Proposition 1. *Let $f_1, \dots, f_d$ be code polynomials (1) of codewords of $\text{Goppa}(\alpha, \Gamma)$, and let $h = f_1 \cdots f_d$. Then $\Pi h' \equiv d\Pi' h \pmod{\Gamma}$. Consequently the $\mathbb{F}_q$ -span R_d of all such products satisfies $\dim R_d \leq dD + 1 - t$ for $d \geq 2$.*

Proof. Let $F_i, \bar{F}_i$ be as in (1) for the i -th codeword. Since F_i' is a square and $\Gamma^2 \mid F_i'$, we have $f_i = \bar{F}_i \sigma_i^2$ for a polynomial σ_i . Differentiating $\Pi = F_i \bar{F}_i$ gives $\Pi' \equiv F_i \bar{F}_i' \pmod{\Gamma}$, because $\Gamma \mid F_i'$. Multiply by $\prod_l \bar{F}_l$ and use $F_i \bar{F}_i = \Pi$:

$$\Pi' \prod_l \bar{F}_l \equiv \Pi \bar{F}_i' \prod_{l \neq i} \bar{F}_l \pmod{\Gamma} \quad \text{for each } i.$$

Summing over i gives $d\Pi' \prod_l \bar{F}_l \equiv \Pi (\prod_l \bar{F}_l)'$. Squares have zero derivative, so $h' = (\prod_i \sigma_i^2)(\prod_l \bar{F}_l)'$, while $h = (\prod_i \sigma_i^2) \prod_l \bar{F}_l$. Multiplying the last congruence by $\prod_i \sigma_i^2$ gives the identity.

The identity is $\mathbb{F}_q$ -linear in h , so it holds on all of R_d . It is one linear condition with values in $\mathbb{F}_q[Z]/(\Gamma)$, that is, t conditions over $\mathbb{F}_q$. These t conditions are linearly independent on the polynomials of degree at most dD , because the map $h \mapsto (\Pi h' - d\Pi' h) \pmod{\Gamma}$ is onto. To see this, take $h = \Gamma u$ with $\deg u < t$, so that $\deg h < 2t \leq dD$. The value of the map is $\Pi \Gamma' u \pmod{\Gamma}$. This ranges over all residues as u does, since $\Pi \Gamma'$ is invertible modulo the square-free Γ . $\square$

Equality $\dim R_d = dD + 1 - t$ held in every instance we computed (Section 6), and it is what [13, Table 5] report.

For a single code polynomial the identity is exact. From $f = \bar{F} \sigma^2$, $\Pi = F \bar{F}$ and $F' = \Gamma^2 \sigma^2$ one gets $\Pi f' + \Pi' f = \Gamma^2 f^2$. This is the binary differential identity of [4, Lemma 3], where it is the starting point of a lower bound discussed in Section 8. With the product rule it gives $\Pi h' + d\Pi' h = \Gamma^2 h \sum_i f_i$ for $h = f_1 \cdots f_d$. This is a second proof of Proposition 1, and it shows that the congruence holds even modulo Γ^2 . The stronger congruence puts no further conditions on R_d , in line with the measured equality.

The consequence for the distinguisher is as follows. The polynomial φ_h lies in R_d . Let the *shortfall* δ be the amount by which the number of forced zeros falls short of $dD_\ell + 1$. Condition (2) says $\delta \leq 0$. When $0 < \delta \leq t$, the quotient of φ_h by the forced zeros ranges over a space of dimension δ . Proposition 1 imposes t conditions over $\mathbb{F}_q$ on this space. Heuristically these conditions leave only zero, so $\varphi_h \equiv 0$ still holds, except with probability about $q^{-(t-\delta+1)}$.

Assumption 2. On binary Goppa keys the conclusion $\varphi_h \equiv 0$ of [13, Theorem 3.2] holds whenever the number of forced zeros is at least $dD_\ell + 1 - t$, that is, whenever $\delta \leq t$.

We tested this directly (Section 6). We built R_d from the secret key and computed, in about 600 runs with t up to 14, the space of nonzero $\varphi \in R_d$ that have all the forced zeros. Its dimension was exactly $\max(0, \delta - t)$ in every non-degenerate run. The binary distinguisher itself accepts Goppa keys up to shortfall t . From shortfall $t + 2$ on it rejects them, as it rejects random matrices. For $d = s + 1$ the relaxed condition reads $k_\ell \leq t(2s + 3 - m)$ in place of $k_\ell \leq t(2s + 2 - m)$. This is worth about 11 bits, and it is what permits $s = 6$.

4. The point at infinity (exact).

Proposition 2. *For every codeword c of the instance, the coefficient of Z^{D_ℓ} in f_c is the parity $\sum_j c_j \in \mathbb{F}_2$. Hence the sum w of all columns of the instance equals $p(\infty)$, the vector of leading coefficients of p . Imposing the conditions of order below s at w forces the top s coefficients of every φ_h to vanish.*

Proof. By interpolation, $f_c = \sum_j c_j \Pi / ((Z - \alpha_j) \Gamma(\alpha_j)^2)$. Expand $\Pi / (Z - \alpha_j)$ in powers of Z . The coefficient of $Z^{n_\ell - 1 - r}$ in f_c is then a fixed linear combination of the sums $\sum_j c_j \alpha_j^i / \Gamma(\alpha_j)^2$ with $i \leq r$, and the sum with $i = r$ has coefficient 1. These sums vanish for $i < 2t$, because they are the parity checks of $\text{Goppa}(\alpha, \Gamma^2) = \text{Goppa}(\alpha, \Gamma)$. This is why $\deg f_c \leq D_\ell$. For $r = 2t$ only $i = 2t$ survives, and

$$\sum_j c_j \alpha_j^{2t} / \Gamma(\alpha_j)^2 = \left(\sum_j c_j \alpha_j^t / \Gamma(\alpha_j) \right)^2 = \left(\sum_j c_j \right)^2.$$

The last step holds because α_j^t equals $\Gamma(\alpha_j)$ plus lower powers of α_j , and the parity checks of the lower powers vanish. For the last claim substitute $Z = 1/u$. Then $u^{dD_\ell} \varphi_h(1/u) = h(u^{D_\ell} p(1/u))$, and $u^{D_\ell} p(1/u)$ equals w at $u = 0$. $\square$

So w is one more point at which conditions can be imposed, and it is worth s zeros in (2). We call the constraint columns together with w the *constraint points*. The extra point matters when the zero count, rather than the kernel dimension, is the binding constraint. This is the case for mceliece460896 only, where it is worth 12 bits. We know of no further points of this kind. For generic z the vector $p(z)$ is not proportional to a binary vector.

Resulting parameters. We minimise the cost of a run, as given by [13, Section 5.3], subject to $\kappa > 0$ and to the relaxed zero count. One formula has to be adapted. The solver first reduces E to a square system. The size of this system is a bound on the rank of E plus $\lceil 2 \log_2 N \rceil$. In [13] the bound is the number of columns of E , which condition (3) makes smaller than N . At our parameters E has about 1.5 times as many columns as rows, so we use the number of rows N as the bound. With the number of columns in its place, the figures of Table 2 would be 0.7 bits higher. The result is Table 2. As in [13], the smallest feasible k_ℓ is optimal, and $d = s + 1$.

4 Key recovery from gradients

4.1 The gradient theorem

Consider a run on a Goppa key in which $\varphi_h \equiv 0$ for every h in the row kernel K . This is guaranteed by (2), or else by Assumption 2. Let y_τ be a column of the instance at which no condition of order

parameter set	(m, t)	ℓ	k_ℓ	n_ℓ	$\log_2 N$	$\log_2$ cost of a run
mceliece348864	(12, 64)	2566	154	922	38.4	94.1
mceliece460896	(13, 96)	3165	195	1443	40.8	99.3
mceliece6688128	(13, 128)	4800	224	1888	42.2	102.4
mceliece6960119	(13, 119)	5197	216	1763	41.8	101.6
mceliece8192128	(13, 128)	6304	224	1888	42.2	102.4

Table 2: Distinguisher parameters with $(d, s) = (7, 6)$ and the four observations of Section 3. $N = \binom{k_\ell}{7}$ is the number of unknowns of a run. For comparison, [13, Table 2] has $(d, s) = (8, 7)$, $k_\ell = 217, 274, 315, 304, 315$ and costs 114.2, 120.3, 124.0, 123.0, 124.0.

1 or higher was imposed. It may be a held-out column, or a column at which only the value condition $h(y_\tau) = 0$ was imposed. For $h \in K$ let $g = \nabla h(y_\tau) \in \mathbb{F}_2^{k_\ell}$ be the vector of first-order Hasse derivatives of h at y_τ . Read g as a message, and let $c = gG$ be the corresponding codeword of the instance, where G is the generator matrix of the instance. Thus $c_j = \langle g, y_j \rangle$.

Theorem 3. *The codeword c lies in $\text{Goppa}(\alpha \setminus \{\alpha_\tau\}, \Gamma \cdot (Z - \alpha_\tau))$. That is, $c_\tau = 0$ and $\sum_{j \neq \tau} c_j / (\alpha_j - \alpha_\tau) = 0$. These are $m+1$ binary linear conditions, so this subcode C'_τ has codimension at most $m+1$ in the code of the instance.*

Proof. By Euler's identity, $c_\tau = \langle g, y_\tau \rangle = d \cdot h(y_\tau)$. This is zero because $h(y_\tau) = 0$ (and for even d it is zero regardless). Next, differentiate $\varphi_h = h \circ p \equiv 0$ at α_τ . The partial derivative $\partial_i h$ is homogeneous of degree $d-1$, and $y_\tau = \lambda_\tau p(\alpha_\tau)$, so

$$0 = \varphi'_h(\alpha_\tau) = \sum_i (\partial_i h)(p(\alpha_\tau)) f'_i(\alpha_\tau) = \lambda_\tau^{-(d-1)} \sum_i g_i f'_i(\alpha_\tau).$$

Hence the code polynomial $f_c = \sum_i g_i f_i$ of c satisfies $f_c(\alpha_\tau) = f'_c(\alpha_\tau) = 0$. By (1), $f_c = \bar{F} \sigma^2$ with $\sigma^2 = F'/\Gamma^2$. Since $c_\tau = 0$, the element α_τ is a simple root of $\bar{F}$. Therefore $f'_c(\alpha_\tau) = \bar{F}'(\alpha_\tau) \sigma(\alpha_\tau)^2$ with $\bar{F}'(\alpha_\tau) \neq 0$, and so $\sigma(\alpha_\tau) = 0$. This means $F'(\alpha_\tau) = 0$, that is, $F(\alpha_\tau) \sum_{c_j=1} (\alpha_\tau - \alpha_j)^{-1} = 0$, and $F(\alpha_\tau) \neq 0$. $\square$

In all our instances the codimension of C'_τ was exactly $m+1$.

The theorem proves containment. One run returns about k_ℓ kernel vectors. That their gradients $\text{span } C'_\tau$ is a genericity statement.

Assumption 3. The gradients at y_τ of $k_\ell + 16$ random elements of the row kernel span C'_τ .

In our experiments (Section 6) the assumption held at every column with one kind of exception. If the column lies on a dual codeword of weight $d+1$ of the instance, the span is $d-1$ dimensions short. This is the mechanism of [13, Appendix A.2]. Such dual codewords do not exist at Classic McEliece sizes: the expected number through a given column is below 2^{-97} . In any case a short span is visible from public data, since the dimension of C'_τ is known.

4.2 The attack

Step 1. Shorten the public code and bring it to systematic form, as in Table 2.

Step 2. Repeat the following until every column of the instance has been treated. Pick H new columns and take a systematic form in which they are non-unit columns. Impose only

the value condition at these H columns. Impose the conditions of all orders below s at all other non-unit columns and at w . Compute $k_\ell + 16$ random row-kernel vectors with a block Krylov solver. Record the span of their gradients at each of the H columns. This step is the entire cost.

Step 3. From the family of subcodes $\{C'_\tau\}$, recover the support of the instance (Section 4.3).

Step 4. Recover the multipliers and the support elements of the ℓ shortened-away coordinates by linear algebra, and verify the key by decoding (Section 4.3).

The number of runs. A column with only the value condition contributes one zero to (2) instead of s . Each such column therefore uses up $s - 1$ zeros of the slack in the zero count. The slack is the number of forced zeros, with w included, beyond what (2) requires (for columns (a)) or beyond what Assumption 2 requires (for columns (b)). In the second case we keep two zeros in reserve, which puts the failure probability estimated in Section 3 at q^{-3} . We take H to be the slack divided by $s - 1$, rounded down. We also require the row kernel to be large enough for its gradients to span C'_τ . The instance needs $\lceil n_\ell/H \rceil$ runs.

Cost. The solver of [13, Section 5.3] uses block size 589. This is more than the $k_\ell + 16$ kernel vectors that one run has to return, so the solver is used as it is. Each gradient costs N operations. Hence the total cost is $\lceil n_\ell/H \rceil$ times that of a run. Table 3 lists the figures. Column (a) keeps every condition and formula of [13] and adds only Theorem 3, Proposition 2 and the finishing phase. Column (b) uses the parameters of Section 3.

parameter set	(a), with $(d, s) = (8, 7)$				(b), with $(d, s) = (7, 6)$			
	k_ℓ	H	runs	$\log_2$ total	k_ℓ	H	runs	$\log_2$ total
mceliece348864	216	9	110	120.8	154	9	103	100.8
mceliece460896	274	4	381	128.9	195	1	1443	109.8
mceliece6688128	314	14	142	131.0	223	8	236	110.2
mceliece6960119	303	11	169	130.3	216	6	294	109.8
mceliece8192128	314	14	142	131.0	223	8	236	110.2

Table 3: The gradient attack: columns treated per run, number of runs and total cost.

4.3 From the subcodes to the key

This finishing phase is ordinary linear algebra over $\mathbb{F}_2$ and $\mathbb{F}_q$. Its largest system has $n_\ell m$ unknowns, which is between 10^4 and $2.5 \cdot 10^4$ for the Classic McEliece parameter sets. It costs about 2^{40} to 2^{44} bit operations and runs in minutes. We describe it in outline. The numbers in square brackets are the dimensions that our implementation printed on a genuine mceliece348864 key ($k_\ell = 154$, $n_\ell = 922$). Each equals the value predicted by the formula next to it. Section 6 lists the other sizes at which every count came out as predicted.

What is known at each column. Regard C'_τ as a space of messages $g \in \mathbb{F}_2^{k_\ell}$. Its annihilator in $\mathbb{F}_2^{k_\ell}$ has dimension $m + 1$ and contains y_τ . The remaining m dimensions come from the second condition of Theorem 3, which reads $\langle g, \psi_\tau \rangle = 0$ with $\psi_\tau = \sum_{j \neq \tau} y_j / (\alpha_j - \alpha_\tau) \in \mathbb{F}_q^{k_\ell}$. Over $\mathbb{F}_q$, a complement V_τ of y_τ in the annihilator is spanned by the m Frobenius conjugates of ψ_τ . Each

conjugate spans a line in $V_\tau \otimes \mathbb{F}_q$. Looking at one column alone, one cannot tell which line belongs to which branch.

The alignment system. The lines are matched across columns by one global linear system. The unknowns are vectors $x_\tau \in V_\tau \otimes \mathbb{F}_q$, one for each column, which makes $n_\ell m$ unknowns. For two messages g, g' with codewords c, c' , define the symmetric bilinear form

$$B_x(g, g') = \sum_{\tau} (c_{\tau} \langle x_{\tau}, g' \rangle + c'_{\tau} \langle x_{\tau}, g \rangle).$$

We require $B_x = 0$. This gives $\binom{k_\ell}{2}$ equations [11781 equations, 11064 unknowns]. Suppose all x_τ lie on the lines of one branch, say $x_\tau = \theta_\tau \psi_\tau$ with scalars θ_τ . A short computation gives $B_x(g, g') = \sum_{\tau} \theta_\tau \lambda_\tau^2 (f_c f_{c'})'(\alpha_\tau)$. By Proposition 1 with $d = 2$, the derivative of a product of two code polynomials is Γ^2 times the square of a polynomial of degree at most $D_\ell - 1 - t$. It follows that $B_x = 0$ when θ lies in a certain GRS code of dimension $n_\ell - (D_\ell - t) = 3t + 1$ whose support is a conjugate of α . We call these solutions the *pure family* of the branch. There is one pure family for each of the m branches. The full solution space also contains solutions that mix several branches. Its dimension is $m(3t + 1) + mt + m - 1$ [3095].

Selecting one branch. The pure family of one branch has to be separated from the rest. For this we use three explicit identities, for $x = \alpha\psi$, $\alpha^2\psi$ and $\alpha^3\psi$. Their forms B_x involve only the known vector w and the power sums $G\alpha^{2^u}$. The identities lead to a small simultaneous-diagonalisation problem whose solutions are the m conjugate directions [a space of dimension $m + 1 = 13$, then $m = 12$ solutions]. We fix one of them, which fixes a branch. Next, let X be the space of all x for which B_x is a diagonal form, enlarged by the two solutions that the identities for $\alpha^2\psi$ and $\alpha^3\psi$ provide [dimension $3863 + 2$]. An element of X that vanishes on $5t + 2$ random columns is supported on the lines of the chosen branch only. Two such elements determine the line of that branch at every column [600 columns after the first, all 922 after the second].

Reading the support. With the lines known, the pure family of the chosen branch [$3t + 1 = 193$] is an explicit GRS code. Take two of its codewords with $3t - 1$ common zeros. Their ratio is a Möbius image of the support, as in [25].

From the support to a key. The support is now known up to a Möbius map and a Frobenius conjugation. This is enough for an equivalent key. GRS codes are invariant under Möbius maps, and conjugate supports give the same Goppa code (Section 2). The instance is therefore the alternant code of degree $2t$ of the recovered support, for multipliers that are found by linear algebra as in [27]. Finally, the mt known columns fix the change of basis between this description and the public parity-check matrix. Applying the change of basis to the other columns of the public matrix returns the support elements of the shortened-away coordinates. This is the completion step familiar from [16].

5 Key recovery from a single run

The gradient attack learns something only at the few columns that a run leaves free. This section reads the support of *every* constraint column from one run. It uses the column kernel of E instead of the row kernel. The starting point is the structural statement behind the key recovery of [13].

5.1 The structure of the interpolation relation

We state the result of [13, Section 7 and Appendix C.1] in the form we use. We refer there for the derivation, which lifts the explicit univariate Hermite interpolation formula through the chain rule. We also take from there the terms canonical relation, no-slack condition and locator block.

Setting. Give the columns of the instance individual multiplicities s_j . Designate a few columns as held-out columns, all of the same multiplicity s' , where $s' - 1$ is a power of two. Unlike in Section 2, a held-out column here does carry conditions: those of all orders below s' are part of E . Let τ be a held-out column. Suppose the value condition at τ is a linear combination of the conditions at all the other columns. Let ρ_τ be the coefficient vector of this combination, normalised so that it has coefficient 1 on the value condition at τ and 0 on the other conditions at τ . It is a column-kernel vector, and it is called the *canonical relation* of τ .

Assumption 4 ([13, Lemma C.1 and Section 7.3]). For each held-out column τ , count the conditions at all other columns. A constraint column of multiplicity s_j counts for s_j , a unit column counts for $d - 1$, and the point w , if it is used, counts for the multiplicity given to it. Suppose this count is exactly $\dim R_d = dD_\ell + 1 - t$. This is the *no-slack condition* of [13] in its form for binary Goppa codes. Suppose also that these conditions are linearly independent as functionals on R_d . Let $\beta_j = \alpha_j^{s'-1}$. Then there are a scalar η_τ and two vectors V_1, V_2 that *do not depend on τ* such that

$$\begin{aligned} \rho_\tau &= \eta_\tau(\beta_\tau V_1 + V_2) && \text{outside the block of } \tau, \\ V_2 &= \beta_j V_1 && \text{on the top } s' - 1 \text{ orders of the block of every column } j. \end{aligned}$$

The top $s' - 1$ orders of the block of j are its *locator block*.

What this gives. Since $s' - 1$ is a power of two, $x \mapsto x^{s'-1}$ is a field automorphism, and β is a conjugate of the support. The two vectors V_1, V_2 therefore show the support element of every constraint column at once, as the ratio of their entries on that column's locator block. We call this the *ratio test*. It is what removes the factor $\lceil n_\ell/H \rceil$ of Section 4. As E is binary, its column kernel is stable under Frobenius, so each of the m branches gives such a pair (V_1, V_2) . The support is only defined up to a Möbius map. Three held-out columns fix this freedom: we normalise their values of β to 0, 1, ∞ . A fourth held-out column selects the branch.

Two remarks from our experiments (Section 6). First, the count must be exact. With fewer conditions there is no relation. With too many conditions, the normalised relations form a family of positive dimension, and this defeats the ratio test as soon as that dimension reaches $s' - 1$. For a generic GRS code the exact count would be $dD_\ell + 1$. For binary Goppa codes it is t less, by Proposition 1 (see also [13, Section 7.3]).

Second, the power-of-two condition admits $s' = 2$, but $s' = 2$ does *not* work on binary Goppa keys. The relations exist and are unique, but they have no common pair (V_1, V_2) . We use $s' \geq 3$. For these values Assumption 4 held in every instance and at every derivative order we examined. A plausible reason for the failure at $s' = 2$ is that R_d contains a t -dimensional part consisting of squares, which first-order conditions cannot see. Indeed Proposition 1 says that $\varphi \bmod \Gamma^2$ is a square, up to the fixed factor Π for odd d .

5.2 The obstacle: local relations, and their closed form

The column kernel of E is enormous, and almost all of it has nothing to do with the code. There are relations among the conditions of a single column. There are also conditions that two columns share, and each shared condition gives a relation. At $(d, s) = (7, 6)$ the shared conditions alone account for two thirds of the top-order conditions of every column. These *local relations* form the space K_{loc} of [13, Section 7.1]. They are the syzygies behind the alternating signs in (4). A random column-kernel vector is a canonical relation plus a large local part, so any use of Assumption 4 must first remove K_{loc} .

The algorithm of [13] removes it implicitly. It solves a fresh copy of the whole system, with the locator condition added, for every column and every candidate value. That is about $n_\ell q$ solves for each guess of the fourth held-out value. To get by with a single run, one needs K_{loc} explicitly. The following proposition provides it.

Proposition 4. *Let $h = \sum_A h_A X^A$ be multilinear of degree d in $X_1, \dots, X_{k_\ell}$, and let $y \in \mathbb{F}_2^{k_\ell}$ have support Σ .*

- (i) *The derivative condition of index S at y is the coefficient of X^S in $h(X + y)$. The conditions of order $d - i$ are the coefficients of the i -th difference*

$$\Delta_y^{(i)} h = \sum_{|B|=d-i} X^B \sum_{U \subseteq \Sigma \setminus B, |U|=i} h_{B \cup U}.$$

Hence the span O_y of the conditions of orders below s_y at y is the span of all coefficients of $\Delta_y^{(i)} h$ with $i \geq e_y := d - s_y + 1$. A column-kernel vector ρ defines on the block of y the functional $h \mapsto \sum_S \rho_{y,S} \partial^{[S]} h(y)$, and this functional does not change when a relation internal to the block is added to ρ .

- (ii) *For two columns a, b and $i \geq e_a, i' \geq e_b$, every coefficient of $\Delta_a^{(i)} \Delta_b^{(i')} h$ lies in $O_a \cap O_b$. Written in the conditions of a it is, for a set C of size $d - i - i'$,*

$$g_{a \leftarrow b}(C) = \sum_{U \subseteq \Sigma_b \setminus C, |U|=i'} \partial^{[C \cup U]} h(a).$$

This is a sum of at most $\binom{|\Sigma_b|}{i'}$ conditions of order $d - i$ at a . The same coefficient can be written symmetrically in the conditions of b . The difference of the two expressions is a column-kernel vector of E supported on the two blocks.

Proof. (i) is the Taylor expansion of a multilinear polynomial at a binary point. Over $\mathbb{F}_2$ we have $h(X + y) = \sum_S X^S \partial^{[S]} h(y)$, and collecting the terms of degree $d - i$ gives the displayed difference.

(ii) The operators Δ_a and Δ_b commute, since both can be read off from $h(X + za + z'b)$. So the coefficients of $\Delta_a^{(i)} \Delta_b^{(i')} h$ are coefficients of an i -th difference along a of some polynomial, and also of an i' -th difference along b of another polynomial. By (i) they lie in both spans. Expanding $\Delta_b^{(i')}$ first, and then reading the coefficient of X^C in $\Delta_a^{(i)}$ of the result via (i), gives the formula. $\square$

Whether these vectors span all of $O_a \cap O_b$ is again a genericity question. Not all pairs (i, i') are needed. Summing the formula of (i) over one further variable shows that the vectors with $i + 1$ in place of i are combinations of those with i whenever $i + 1$ is odd. So above the minimal values (e_a, e_b) only even values of i, i' can contribute anything new. In our computations (Section 6) the vectors with minimal (i, i') , together with those at even values, spanned $O_a \cap O_b$ exactly whenever the weights of a and b are not close to d . At $k_\ell \geq 154$ the weights are about $k_\ell/2$. For example, at

$(d, s) = (7, 6)$ and two columns of multiplicity 6 we have $\dim O_a \cap O_b = \binom{k_\ell}{3}$. This space is spanned by the vectors with $(i, i') = (2, 2)$, for which $|C| = 3$, and those with $(4, 2)$ and $(2, 4)$, for which $|C| = 1$.

Assumption 5. At the parameters of Table 4, the column kernel of E is spanned by three kinds of relations: relations internal to one block, the two-block relations of Proposition 4, and the relations lifted from univariate interpolation on R_d . Of the last kind there are ms' in the setting used below.

This is the analogue, for the column kernel, of Assumption 1. It is the kernel-decomposition hypothesis of [13, Appendix C.2] with K_{loc} made explicit. On the toy key it fails by a hundred-odd relations that involve columns of weight close to d . The algorithm below tolerates them, and they cannot occur at real size.

5.3 The algorithm

Setting up the instance. Take the instance of Table 2, with w as a constraint point of multiplicity 6. Choose four held-out columns and give them multiplicity $s' = 5$. Then lower as many other columns from multiplicity 6 to 5 as are needed to make the count of Assumption 4 exact. Table 4 lists this number. It is computed as follows. Start from the surplus of conditions over $dD_\ell + 1 - t$ when all columns and w have multiplicity 6. Subtract 9: the relation of a held-out column does not use that column's own block of 5 conditions, and each of the four held-out columns has already been lowered by one. For `mceliece460896` nothing is left, so the count is exact with no column to lower. In our experiments w carried only its value condition, that is, multiplicity 1. Four of the five parameter sets have room for the same choice, which lowers the number in Table 4 by 5. Only `mceliece460896` needs w at full multiplicity.

Only constraint columns can be read, because the conditions at unit columns are trivial. The completion step of Section 4.3 recovers the unit columns and the shortened-away coordinates afterwards, and it needs the support elements of somewhat more than mt columns. For this reason we keep a handful of further columns out of the information set.

- Step 1.** Perform one run, organised so that it returns $r \approx 200$ random vectors of the *column* kernel of E . The matrix and the solver are the same as before, and r is below the block size of the solver. This step is the entire cost.
- Step 2.** Remove the local relations. For each constraint column j , read the r vectors as functionals on the block of j (Proposition 4(i)). Reduce them modulo the two-block vectors $g_{j \leftarrow b}$ for all other blocks b . Reduce them also modulo the conditions that carry no support information: the value condition at a held-out column, and the orders up to $s_j - s'$ at the other columns. What remains of each vector at block j lies in a small space, of dimension $(s' - 1)m$: one line for each branch and each order of the locator block.
- Step 3.** Find the branches. Discard the combinations of the r vectors that vanish at all four held-out blocks, since these are local relations. This keeps $r' = ms'$ combinations. Now look for pairs (V_1, V_2) of such combinations. For each of the three held-out columns τ that are normalised to $\beta_\tau = 0, 1, \infty$, require that $\beta_\tau V_1 + V_2$ has no component on the derivative conditions at τ . This is a binary linear system. Its solution space has dimension m over $\mathbb{F}_2$: one dimension for each branch.
- Step 4.** Select a branch. At the fourth held-out column the m solutions give two binary matrices, A_1 from the V_1 -parts and A_2 from the V_2 -parts. They satisfy $A_2 = TA_1$ for a unique $m \times m$ binary matrix T . The eigenvalues of T in $\mathbb{F}_q$ are the m conjugates of the normalised β of that column. A left eigenvector of T selects the branch. Nothing is guessed.

Step 5. Read the support. For every other constraint column j , the selected branch gives two vectors $v_1, v_2 \in \mathbb{F}_q^{(s'-1)m}$ with $v_2 = \beta_j v_1$. Read β_j as their ratio. Then finish as in Section 4.3: multipliers, completion, and a decoding check.

Cost. Step 2 is a sparse elimination for each column. Its generators are the two-block vectors through the column. There are $\binom{k_\ell}{3} + 2k_\ell$ of them for each of the P other constraint points, and each has weight about $\binom{k_\ell/2}{2}$. Done with a block Krylov method, this costs about 2^{65} for one column. For the whole instance it costs about 2^{75} at mceliece348864 and about 2^{82} for the sets with $m = 13$. This is at least 17 bits below the cost of the run. Steps 3–5 are negligible. Table 4 gives the cost of the run.

parameter set	k_ℓ	n_ℓ	columns to lower	(a)	(b)
mceliece348864	155	923	40	114.3	94.3
mceliece460896	195	1443	0	120.3	99.3
mceliece6688128	224	1888	35	124.0	102.4
mceliece6960119	216	1763	25	123.0	101.6
mceliece8192128	224	1888	35	124.0	102.4

Table 4: The single-run attack: $\log_2$ cost of the single run. At most a few tenths of a bit should be added for the extra constraint columns and for Step 2. Column (a) uses the conditions and formulas of [13] plus Proposition 2. Column (b) uses the parameters of Section 3, and k_ℓ , n_ℓ and the number of columns to lower from multiplicity 6 to 5 refer to it.

6 Experiments

Each item below states what was computed and what was observed. Everything was done with our implementation (SageMath and M4RI, on a single machine). The sizes are chosen so that every item can be re-run in minutes to hours. Several tests work in the *univariate model*: they build the space R_d from the secret key and test a statement about polynomials in R_d directly, without forming the matrix E . The other tests run the binary algorithm on E itself.

6.1 The statements of Section 3

Propositions 1 and 2. We checked Proposition 1 and the equality $\dim R_d = dD + 1 - t$ on 10 codes with $d = 2, \dots, 5$ and $m = 4, \dots, 8$, with full and partial supports, shortened and not. The identity held on all products, and the dimension defect was exactly t every time. The same equality appeared in every later experiment that builds R_d , up to $d = 8$ and $t = 14$. For Proposition 2, the leading coefficient equalled the parity for every row of every key tried. In the univariate model the extra point removed exactly s dimensions, and the acceptance boundary of the binary algorithm moved accordingly.

Assumption 2. We tested the assumption in the univariate model on 621 runs. They cover 14 combinations of (m, t, d, s) with $t \leq 14$, and shortfalls from -2 to $t + 3$. A run is *non-degenerate* if the instance is large enough for the products of code polynomials to reach the generic dimension of R_d . In all 207 non-degenerate runs, the space of admissible φ had dimension $\max(0, \delta - t)$. There was one rejection at shortfall exactly t in about thirty such runs, which is consistent with the q^{-1}

estimate, and there was none below t . We also ran the binary algorithm on keys with $m = 6, 7$ and $t = 3, \dots, 6$. Goppa keys were accepted up to shortfall t , with kernels of dimension several hundred to several thousand, while random matrices of the same shape were rejected. From shortfall $t + 2$ on, Goppa keys were rejected too.

Assumption 1. We computed the row kernel on random and Goppa instances with $k_\ell = 16, \dots, 28$ and $d = 4, 5$, including well past the boundary of (3). Its dimension was never below (4). It equalled (4) or exceeded it by a small amount that shrinks as k_ℓ grows. Held-out random columns were rejected in at least 90% of the trials as long as the kernel was nonzero. They were never rejected once the kernel was zero.

6.2 The gradient attack (Section 4)

End to end on the toy key. The toy key is the smallest binary Goppa code that has a nontrivial kernel inside the range where $\varphi_h \equiv 0$ holds. We used $(d, s) = (5, 4)$ with 35 constraint columns. The matrix of a run is 115640×98280 , and a run takes three minutes with M4RI. We ran the attack on two keys with 64 runs each, one for each column. The results were as follows.

- The value of every kernel vector at its held-out column was 0.
- Every one of about 64000 gradients lay in C'_τ . This was checked against the secret key.
- The gradients spanned C'_τ , of dimension $k - m - 1 = 21$, at 58 of the 64 columns. The other six columns form the support of the unique dual codeword of weight $6 = d + 1$. There the gradients spanned $17 = 21 - (d - 1)$ dimensions. Holding out two of these columns together repaired this.
- The finishing phase then recovered a key that rebuilds the public code and decodes random errors of weight t .

We also ran two variants. The first uses three value-only columns per run together with the point w , which takes 22 runs instead of 64. The second is a shortened instance ($m = 6$, $t = 5$, $k = 34$ shortened to $k_\ell = 27$), including the completion of the seven shortened-away coordinates. A second implementation, written independently, repeated the gradient checks on six further keys with 19 runs.

The finishing phase at full size. Steps 3 and 4 of Section 4 do not need a run if the subcodes C'_τ are computed from the secret key. They can therefore be tested at real size.

- On a genuine mceliece348864 key ($m = 12$, $t = 64$, $n = 3488$, shortened to $k_\ell = 154$, $n_\ell = 922$), every dimension of Section 4.3 came out at its predicted value. The recovered key rebuilt the public code on all 3488 coordinates and decoded random errors of weight 64. This took 40 minutes on one core.
- On a genuine mceliece8192128 key ($m = 13$, $t = 128$, $n = 8192 = q$, $k_\ell = 223$, $n_\ell = 1887$, which is also the shape of the shortened mceliece6688128), all counts were again as predicted: 24753 equations of rank 17850, hence 6681 solutions, and a GRS family of dimension $385 = 3t + 1$. The public code was rebuilt on all 8192 coordinates. This took 25 minutes.
- The same held at eleven toy sizes with $m = 6, \dots, 10$ and $t = 5, \dots, 10$.

6.3 The single-run attack (Section 5)

On the toy key we use $d = 5$, four held-out columns of multiplicity $s' = 3$, multiplicity 4 at the other constraint columns (the *regular* columns), and the point w with multiplicity 1. This meets the exact count $250 = dD + 1 - t$. The matrix is 98280×119485 .

Assumption 4 in the univariate model. We used keys with (m, t) from $(6, 4)$ to $(7, 10)$ and $s' = 3, 5$. The relations were unique. They had the stated form at every order of every block, and the ratio test accepted the true support element and no other value at every column. With one condition too few there was no relation. With t conditions too many, every field element passed the test. With $s' = 2$ the relations were unique but admitted no common (V_1, V_2) , on every key tried. Their cross-ratio differs from that of the support.

Assumption 4 on the binary matrix. On the toy matrix we ran the test of [13] exhaustively over $\mathbb{F}_{64}$. With three held-out columns imposed, the set of accepted values at each of the 33 other constraint columns was exactly the set of the m conjugates of the true normalised value. The values of the fourth held-out column that are consistent with the constraints were exactly its m conjugates. With one of them imposed, every column accepted the true value alone. Two information sets gave all 64 support elements, unique multipliers, the public code and a working decoder.

Proposition 4 and Assumption 5. On the toy matrix we computed the exact dimension of $O_a \cap O_b$ for all 630 pairs of constraint columns. It is $28 = k$ for two regular columns and 1 for a regular and a held-out column. We also computed the internal relations of every block. For 84 pairs we checked that the vectors of Proposition 4 are column-kernel vectors and that they span the intersection exactly. We repeated this for random columns at $(d; s_a, s_b) = (6; 5, 5)$, $(7; 6, 6)$ and $(7; 6, 5)$ with $k_\ell = 20, 22$, with the same result. The counts are those quoted after the proposition. At $(7; 6, 6)$ and $k_\ell = 22$ the intersection has dimension $1540 = \binom{22}{3}$. The vectors with $(i, i') = (2, 2)$ give 1496 of these, and the vectors with $(4, 2)$ and $(2, 4)$ give the remaining $2k_\ell$. At $(7; 6, 5)$ the dimension is $231 = 208 + k_\ell + 1$. The column kernel of the toy matrix exceeds the span of Assumption 5 by 122 dimensions. All of them can be attributed to columns of weight close to $d = 5$. One column of the toy instance has weight 7.

The algorithm end to end. We ran the algorithm of Section 5 from $r = 200$ random column-kernel vectors and the closed forms only, on two keys and two information sets each. We observed $r' = ms' = 18$ informative combinations (24 on the key with the weight-7 column) and exactly $m = 6$ solutions in Step 3. In Step 4, $A_2 = TA_1$ held exactly, and the characteristic polynomial of T equalled the minimal polynomial of the true normalised fourth value. All 32 columns that a run can read were read correctly in every execution. The excess relations of the toy size appear as free directions at some columns, and the algorithm carries them along. The multipliers were unique, the public code was rebuilt and errors were decoded.

7 A challenge instance: TII-253

The challenge. In 2023 the Technology Innovation Institute published a series of McEliece key-recovery challenges [17]. They are public keys of binary Goppa codes with m between 5 and 10. Each is labelled by a nominal security level that the organisers assigned to it, and the labels run from 22 to 255. The labels are not decoding costs, and these codes are far smaller than anything

proposed for use. Standard ISD software on a single core very quickly decodes a ciphertext for the instance labelled 255 [6]. The instances are of interest only as tests of key-recovery algorithms. During the competition the instances up to label 68 and one at label 83 were solved, by an algebraic attack on the codes with $t = 2$ and by an optimised support-splitting search [5, Section 1.2]. In 2026 three lines of work went further. Bardet, Lemoine and Tillich [5] solved the high-rate instances 89, 125, 166 and 210, and Briaud et al. [8] added 119 and 248. Hemmert [14] and then Saarinen [21] used the higher-order-vanishing key recovery to solve 37 instances with labels between 83 and 252, some of them in minutes. These include the six just mentioned. The labels say little about structural hardness. The cost of those attacks is governed by how small a degree suffices for the dual code, which favours large m relative to t . About 130 instances above label 70 remained open. We took label 253, one of the highest. Saarinen [23] has solved label 254, with $m = 8$, $t = 12$, $n = 223$, by a two-anchor variant of the key recovery of [13]. Instance 253 is not among the challenges that [8, Appendix A] lists as within reach of their attack. It has $m = 8$, $t = 9$, $n = 214$, $k = 142$, and the field is $\mathbb{F}_{2^8} = \mathbb{F}_2[a]/(a^8 + a^4 + a^3 + a^2 + 1)$.

Parameters of the attack. For the single-run attack the cost is set by the smallest shortening at which two things hold: the exact count of Assumption 4 can be met, and the row kernel is nonzero. For this instance there is exactly one choice. It is $(d, s) = (6, 5)$ and $k_\ell = 46$, hence $n_\ell = 118$. All 72 non-unit columns have multiplicity 5, four of them are held out with $s' = 5$, and the point w is used with multiplicity 1. Then $\kappa = 112\,098$. The count is $586 = dD_\ell + 1 - t$ with nothing to spare, and $k_\ell = 45$ or 47 fails one of the two conditions. The matrix of a run has $N = \binom{46}{6} = 9\,366\,819$ rows. As in Section 3 we keep only the derivative orders 4 and 2 at each column. This leaves $11\,823\,841$ columns and about $3.6 \cdot 10^9$ nonzero entries. The columns of order 4 alone would not be enough. They are linearly dependent among themselves, by a rank defect of the inclusion matrices involved, and order 2 supplies exactly the missing conditions.

Implementation. The kernel computation is a block Lanczos iteration [19] of block width 256 on $E^\top E$. The matrix E is never stored. It is applied on the fly. For each constraint column the rows are enumerated as bit patterns, and the entries follow from the definition of E : a row A meets the conditions $S = A_{\text{out}} \cup S_{\text{in}}$ with $A_{\text{out}} = A \setminus \Sigma_j$ and $S_{\text{in}} \subseteq A \cap \Sigma_j$. Only seven block vectors of 11.8 million 256-bit entries are held in memory, about 3 GB. One product by E and one by $E^\top$ take about 0.8 s each on 32 threads.

For Step 2 of the algorithm we work with the coordinates of the kernel vectors on each block, not with functionals of length N . The relations internal to one column are obtained exactly. We split that column's conditions according to the part of the monomial that lies outside the column's support. This makes the matrix of the column block diagonal, with a few thousand small inclusion-matrix blocks whose kernels depend only on the weight of the column. The two-block relations are the closed forms of Proposition 4. Reducing the 256 kernel vectors modulo the resulting generators, about 96 000 of them in 179 000 coordinates, is one dense elimination per column with M4RI.

A run reads the non-unit columns only. Two executions with different information sets and different fourth held-out columns therefore read all 118 columns of the instance. The rest is the finishing algebra of Section 4.3. The challenge asks for the key in Goppa form. To obtain it we search over the $q + 1$ cosets of the Möbius group modulo affine maps for the frame in which the degree- $2t$ multipliers are the inverse squares of a polynomial of degree t .

What happened. The two kernel computations converged after 35 903 and 35 902 iterations. This is the rank of E divided by the block width, as expected. Each returned 256 vectors ρ with

$E\rho = 0$ verified. In both executions the reading went exactly as Section 5 predicts for $m = 8$ and $s' = 5$.

- The images at the regular columns had dimension $(s' - 1)m = 32$. A few columns (seven in one execution, four in the other) had more and were read with free directions, as on the toy key.
- The four held-out columns saw exactly $ms' = 40$ independent combinations.
- The three normalised held-out columns left exactly $m = 8$ pairs (V_1, V_2) , one for each branch.
- The fourth held-out column gave $A_2 = TA_1$ exactly, with an irreducible characteristic polynomial of degree 8.
- All 68 regular columns were read.

The two executions selected different branches. After aligning them by the Frobenius power that relates the branches, they agreed on all 26 columns that they share. The degree-18 multipliers of the 118-column instance were unique and reproduce the instance. The completion gave the 96 shortened-away coordinates and reproduces the public code on all 214 coordinates. The Goppa form was found. The resulting pair (L, g) is listed in Appendix A. It generates the public code, decodes random errors of weight 9, and is accepted by the organisers' verification script.

Cost. Each kernel computation took 21.7 hours with 32 threads, which is about 700 core-hours. We used a shared server with a 64-thread Intel Xeon Platinum 8488C and 247 GB of memory. Part of the second computation ran on a 96-thread AMD EPYC 7B13 cloud instance at a similar rate and gave a byte-identical result. Step 2 took two hours per execution on the same server, and everything else took seconds. From start to verified key took two and a half days of wall-clock time. No GPU was used. In the bit-operation model of the rest of this paper, one such kernel computation is roughly 2^{62} . For comparison, the remaining open instances with $m = 8$ have matrices four to eight times smaller by this measure, and the instances with $m = 9, 10$ have matrices thirty to several thousand times larger.

What it adds. The instance is a toy, and solving it says nothing by itself about Classic McEliece. What the computation adds is a test on a key that we did not generate. It tests Assumptions 4 and 5 and the dimension counts of Steps 2–4 of Section 5, at $k_\ell = 46$ rather than 28 and with $m = 8$. All of them held exactly. It does not reach the $k_\ell \geq 154$ of the Classic McEliece parameter sets, where the same statements remain assumptions.

8 Relation to the other key-recovery proposals

Besides the two algorithms in this paper, three key-recovery algorithms that build on the GIJS distinguisher have been described. They are the one added by GIJS themselves [13, Section 7], Vedenev's [26], and Saarinen's specification and costing of Vedenev's route for binary Goppa codes [22]. A note by Apon [4] analyses the reconstruction step of Vedenev's algorithm. All start from the same jet matrix E of one shortened instance. They differ in three respects: which kernel of E they read, what they read from it, and how they resolve the Frobenius ambiguity, that is, the m conjugate curves through the binary points. These differences are what separates their costs. Table 5 summarises them, and the rest of this section explains the entries.

	kernel of E	what is read	Frobenius branch fixed by	large solves per key	search after the solves
GIJS [13, §7]	column	canonical relation	guess of the 4th held-out value	$R_\ell(n_\ell+1)(q+1)$ over $\mathbb{F}_q$	none
Vedenev [26], Saarinen [22]	row	osculating flags at c columns	quadratic separation, then enumeration	$c \geq 2t+3$ [4] (+ batches)	$m^{c-1}q^{c-2}$ guesses
Section 4	row	subcode C'_τ at every column	alignment system (§4.3)	$\lceil n_\ell/H \rceil$ $\approx 100\text{--}1400$	none
Section 5	column	canonical relation modulo K_{loc}	$m \times m$ eigenvalue problem	1–2	none
		$\log_2$ cost, mceliece348864	$\log_2$ cost, five sets	run on binary Goppa	
GIJS [13, Table 3]	130.4		130–149	toy key (Sec. 6)	
Saarinen [22, Table 4]	146.3 / 172.1		146–208 / 172–235	no ([26]: $\mathbb{F}_4$ -Goppa)	
Section 4	100.8 (120.8)		101–110 (121–131)	toy key, finish at full size	
Section 5	94.3 (114.3)		94–102 (114–124)	toy key, TII-253 (Sec. 7)	

Table 5: Key-recovery algorithms built on the GIJS distinguisher. Costs are $\log_2$ bit operations as reported by each source. For GIJS this is their Table 3. For Saarinen these are the “projective nested” and “affine dense” totals for Vedenev’s route. They assume Vedenev’s count $c \leq 9$. With the bound $c \geq 2t+3$ of [4] the guessing stage alone exceeds 2^{1500} . Ours are given with the parameters of Section 3 and, in parentheses, with the conditions and formulas of GIJS. All algorithms use $\mathbb{F}_2$ for the large solves except the locator systems of GIJS.

GIJS [13, Section 7 and Appendix C]. This is the algorithm whose structural statement is Assumption 4. It fixes three held-out values and guesses the fourth, for which there are $q+1$ candidates. For each guess, each column j and each candidate value x , it performs one sparse linear solve over $\mathbb{F}_q$. The solve decides whether the affine space cut out by the held-out conditions contains a pair with $V_2 = x^{s'-1}V_1$ on the locator block of j . Local relations never have to be written down, because each test is a consistency question for the whole system. The price is the number of solves. There are $(n_\ell+1)(q+1)$ solves for each shortened instance, and $R_\ell = 4$ or 5 instances are needed to cover the support. Each solve is about twice the size of a distinguisher run and is over $\mathbb{F}_q$, which is charged a factor m^2 . Their Table 3, which already uses the sharper kernel count of Section 3, gives $2^{130.4}$, $2^{149.0}$, $2^{141.3}$, $2^{140.5}$, $2^{141.6}$. Their Remark 7.3 records the reductions proposed here: sampling the column kernel once, explicit local relations, and $\mathbb{F}_2$ throughout. It anticipates that these bring key recovery to the cost of distinguishing, which is what Section 5 establishes. The end-to-end binary Goppa test reported in [13, Appendix C.2] is the one of Section 6.

Vedenev [26] and Saarinen [22]. Vedenev reads the row kernel, as Section 4 does, but he reads more at fewer columns. At a held-out column y_i the kernel forms determine the tangent line of the curve, and our Theorem 3 is this first-order case. Recursively, through higher-order chain rules, they also determine the whole osculating flag $V_i^{[1]} \subset V_i^{[2]} \subset \dots$ of the curve at α_i . This requires that the m Frobenius branches have first been separated, which is done with the quadratic parts of the forms with vanishing gradient. By counting equations, Vedenev expects flags at $c = \lceil (k_\ell(D_\ell+1) - 1)/\binom{k_\ell}{2} \rceil$ columns to determine the curve. This gives $c \leq 9$ for the Classic McEliece sets, a count that is revisited below. The reconstruction works only after the support elements of $c-2$ of those columns and the relative Frobenius twists have been guessed. There are $m^{c-1}q^{c-2}$ possibilities, and each is followed by a linear solve in $k_\ell(D_\ell+1)$ unknowns. Vedenev ran this end to end on Goppa codes over $\mathbb{F}_4$ and conjectures that it carries over to $\mathbb{F}_2$.

Saarinen [22] writes the binary version out in full. He adds heterogeneous multiplicities and

reduced derivative orders, both of which are also used here, a matrix-free block-Wiedemann cost model, and a third normalising column. He tallies $2^{146}\text{--}2^{208}$ for the five sets, or $2^{172}\text{--}2^{235}$ with two normalising columns. The guessing stage dominates. The kernel computations alone come to $2^{114.4}$ for mceliece348864 at the k_ℓ of GIJS, in agreement with GIJS Table 2 and with our column (a). He is careful to list what is unverified: the binary Krylov yield, that the kernel forms give complete flags, branch synchronisation across columns, and binary reconstruction. In the first-order case these are the statements measured in Section 6: the dimension $m + 1$ of the gradient annihilator, the alignment system, and end-to-end recovery. The eigenvalue step of Section 5 is a constructive branch synchronisation.

Apon’s lower bound [4]. Vedenev’s count of c assumes that the linear equations obtained at different held-out columns are independent. Apon shows that they are not. He grants the algorithm the true flags and the true support elements at c columns, and asks when the flag conditions determine the curve p up to a scalar. In experiments on twelve binary Goppa codes with $m = 8, 9, 10$ and $t \leq 12$, this happened at $c = 2t + 3$ columns and not before. Vedenev’s formula gives between 5 and 20 for the same codes. Apon then proves that $c \geq 2t + 3$ is necessary for every binary Goppa code, under a mild nondegeneracy condition. The proof starts from the identity $\Pi f' + \Pi' f = \Gamma^2 f^2$ of Section 3. With it he builds an explicit family of polynomial vectors, of dimension $2t + 3$ modulo p , on which all the flag conditions at one column reduce to a single evaluation condition. Each held-out column therefore removes at most one dimension of the family. The reason is that a flag records only spans of derivative vectors, not the vectors themselves.

For the cost of Vedenev’s route this changes the exponent. The number of guesses $m^{c-1}q^{c-2}$ is at least q^{2t+1} . This is 2^{1548} for mceliece348864 and 2^{2509} or more for the other sets, in place of the $2^{146}\text{--}2^{235}$ tallied in [22] with $c \leq 9$. Apon states that his result concerns this reconstruction step only, and not every use of the kernels of the distinguisher. The two attacks of this paper do not reconstruct p from flags at a few columns. The gradient attack uses first-order data at all n_ℓ columns, and n_ℓ is about seven times $2t + 3$ for the Classic McEliece sets. It then finds the support through the alignment system and a GRS code, not through p . The single-run attack reads the support elements directly as ratios.

Comparison. The two row-kernel routes trade against each other. Vedenev reads complete flags at c columns but pays q^{c-2} afterwards, because c local flags do not fix the global Frobenius labelling, and by [4] c cannot be small. Section 4 reads only first-order data, but at every column. The labelling is then a linear problem, at the price of $\lceil n_\ell/H \rceil$ solves. The column-kernel routes avoid both costs, because one canonical relation already speaks about every column in one consistent branch. GIJS pay instead for not separating the canonical relation from K_{loc} . Section 5 removes that cost by Proposition 4.

Saarinen’s separate paper [21] improves Hemmert’s higher-order-vanishing key recovery [14]. That is a different mechanism, based on low-degree forms vanishing on the dual code, and it is not competitive for the Classic McEliece sets. It is what solved the TII instances up to label 252 discussed in Section 7.

9 Storage, and the cost model

Storage. The runs of Section 4 are performed one after the other, so the storage of either attack is that of one run. The storage formula of [13, Section 5.3] gives 2^{57} to 2^{62} bits at the parameters of Table 2. But 96% of this is the preconditioner of the particular solver chosen there. A run

organised for low storage need not store E at all. The action of E on a vector can be computed column by column from the k_ℓ -bit columns of the instance, in fewer word operations than a stored sparse product would take. A plain block Wiedemann iteration then holds essentially its block of vectors. This is $2^{47.8}$, $2^{50.5}$, $2^{52.1}$, $2^{51.7}$, $2^{52.1}$ bits for the five sets, or 28 to 540 TiB. Half of it is accessed at random. The operation count in the same model is no greater. The finishing phase needs under 2 GB (measured).

Data movement. The operation count says nothing about where the operands come from. Here that is the larger part of the real cost. Take mceliece348864 at the parameters of Table 2. The formulas of [13, Section 5.3] give $2^{94.1}$ bit operations. Of these, $2^{92.5}$ are products of E and $E^\top$ with single vectors, $2^{92.7}$ are products with the sparse preconditioner of that solver, and $2^{92.4}$ are everything else. Sparse products thus make up 71% of the count, and 77% to 80% for the other four sets. In a sparse product over $\mathbb{F}_2$, one operation adds one bit of the input vector to an accumulator, and the matrix dictates which bit. The entries of E can be generated on the fly, but the vector bits have to be fetched. The products with E and $E^\top$ are needed however the run is organised. On mceliece348864 they alone move $2^{92.5}$ bits between the units that do the additions and the array that holds the vectors. With the preconditioner of that solver it is $2^{93.6}$ bits, so roughly 2^{93} either way. The array has $2^{47.8}$ bits in the low-storage organisation above, which has no preconditioner, and more in the solver of [13]. For the other sets the products with E and $E^\top$ move 2^{98} to 2^{101} bits, and the arrays have $2^{50.5}$ to $2^{52.1}$ bits. The access pattern follows from the definition of E : row A meets the conditions (j, S) with $S \subseteq A$. It is known in advance and is the same in every pass, but we know of no layout of the vectors that makes it local. A count of bit operations charges such an addition the same as the addition of two bits that sit in adjacent registers.

Charging for memory. The Classic McEliece security guide [2] compares attacks after multiplying the time by $M^{1/3}$ or by $M^{1/2}$, where M is the memory. It tabulates ISD in both models: $2^{156.4}$ – $2^{311.9}$ with $M^{1/3}$ and $2^{158.6}$ – $2^{315.6}$ with $M^{1/2}$. With M as above, the gradient attack of column (b) of Table 1 becomes $2^{116.7}$ – $2^{127.5}$ with $M^{1/3}$ and $2^{124.7}$ – $2^{136.2}$ with $M^{1/2}$. The single-run attack of column (b) becomes about 2^{110} – 2^{120} and 2^{118} – 2^{128} . The columns (a) of the gradient attack also stay below ISD charged the same way in every case. The closest case is mceliece348864 with $M^{1/2}$, where the margin is still more than ten bits. What no operation count captures is that a run is a chain of 2^{32} to 2^{36} mutually dependent passes over this memory.

Comparison with a cipher key search. The claimed levels 2^{143} , 2^{207} and 2^{272} are the numbers of bit operations that NIST estimates for a key search on AES-128, AES-192 and AES-256. Table 1 sets bit operations against bit operations. In three respects this flatters the attack.

- A key search needs almost no memory and no communication. Its operations act on bits that are already in place, and it splits into independent units. The hardware that stores the vectors of a run could instead hold a very large number of cipher circuits working in parallel, none of which waits for data. In the models above, the margin between the single-run attack on mceliece348864 and 2^{143} is 49 bits in Table 1, 33 bits with $M^{1/3}$ and 25 bits with $M^{1/2}$. For the other four sets the margin with $M^{1/2}$ is between 82 and 145 bits.
- A key search amortises over targets. With 2^{40} target keys of a 128-bit cipher, one of them is found after about 2^{88} encryptions. The attacks of this paper share nothing between keys: every public key is a separate linear-algebra problem.

- A key search is sped up by Grover’s algorithm. The attacks of this paper are not searches, and we know of no quantum algorithm that speeds up the kernel computation.

We keep the bit-operation figures in the tables because they are what [13] and the other tallies [22] report. The memory-charged figures are the more meaningful ones.

10 Status of the assumptions, and consequences for parameters

Assumptions. Table 6 lists what each column of Table 1 rests on and how far each assumption was tested. Assumptions 1 and 2 are worth about 16 bits together (Section 3). The existence of the attacks does not depend on them. Assumption 3 was tested at toy size only, because testing it needs a run. Assumptions 4 and 5 and the counts of Steps 2–4 of the single-run attack were not tested at Classic McEliece size for the same reason: they concern the output of a run.

figure	rests on	tested
gradient attack, (a)	conditions of [13], Theorem 3, Proposition 2 Assumption 3 dimension counts of Section 4.3	proved toy size full size
columns (b), in addition	Assumption 1 Assumption 2	up to $k_\ell = 28$ up to $t = 14$
single-run attack, in addition	Assumptions 4, 5, counts of Steps 2–4	toy size, TII-253

Table 6: What the figures of Table 1 rest on, and the largest size at which each ingredient was tested. TII-253 has $k_\ell = 46$.

If an assumption failed, the failure would be visible from public data: a wrong dimension, no eigenvalue in $\mathbb{F}_q$, an inconsistent ratio, or a key that does not rebuild the code. The remedy would be another shortening or another held-out column. This means more runs, not a different attack.

Dependence on the parameters. The cost of a run depends on m and t only. It is determined by the smallest k_ℓ with $\kappa > 0$ at the smallest feasible (d, s) . It does not depend on n , as long as $k = n - mt \geq k_\ell$. At fixed m and fixed (d, s) it grows slowly with t : about 4 bits for 50% more t at $m = 13$. It is not monotone in t , however. A larger t relaxes the zero count $k_\ell \leq t(2s + 3 - m)$ and can make a smaller (d, s) feasible. The cost then drops, by up to ten bits in Table 7, as at $m = 13$ between $t = 64$ and $t = 96$. What raises the cost is m . The zero count needs s to be about $m/2$, and each unit of s costs twelve bits or more. Table 7 shows the result: about eight bits per unit of m . At fixed m the cost is polynomial in t , because (d, s) is bounded, $N = \binom{k_\ell}{d}$, and k_ℓ is roughly proportional to t . In m it is exponential. It is therefore quasi-polynomial in n for families of codes with $m = O(\log n)$, such as $n = 2^m$, and it is not for families in which m grows faster.

Key sizes. A larger m does not require a much larger key, because nothing forces $n = 2^m$. Four of the five Classic McEliece sets have $n < 2^m$, and the cost of the attack does not depend on n . The public key has $mt(n - mt)$ bits and a ciphertext has mt bits. The support can be stored as $(\alpha_1, \dots, \alpha_n)$ in mn bits, instead of as control bits for a permutation of all of $\mathbb{F}_q$, and polynomials are then evaluated at these n elements only. At fixed n and t all of these sizes grow at most linearly in m . Raising m at fixed n and t lowers the dimension $k = n - mt$ and with it the cost of ISD, so n has to grow as well. If t and the rate k/n are kept, then mt and k both grow in proportion to m , and the public key grows like m^2 .

m	$t = 64$	$t = 96$	$t = 128$	$t = 160$
12	94.1 (7,6)	98.4 (7,6)	101.5 (7,6)	103.9 (7,6)
13	106.4 (8,7)	99.3 (7,6)	102.4 (7,6)	104.7 (7,6)
14	107.3 (8,7)	112.2 (8,7)	115.7 (8,7)	118.4 (8,7)
15	119.6 (9,8)	125.1 (9,8)	129.0 (9,8)	119.2 (8,7)
16	131.9 (10,9)	126.0 (9,8)	129.8 (9,8)	133.0 (9,8)
17	132.8 (10,9)	138.9 (10,9)	143.2 (10,9)	146.6 (10,9)
18	145.1 (11,10)	151.8 (11,10)	144.1 (10,9)	147.5 (10,9)
20	158.4 (12,11)	165.5 (12,11)	158.3 (11,10)	162.0 (11,10)
22	171.6 (13,12)	179.4 (13,12)	185.0 (13,12)	176.5 (12,11)
24	196.3 (15,14)	193.2 (14,13)	199.3 (14,13)	203.9 (14,13)
26	209.7 (16,15)	207.1 (15,14)	213.6 (15,14)	218.5 (15,14)
28	223.0 (17,16)	233.2 (17,16)	227.9 (16,15)	233.2 (16,15)
30	248.0 (19,18)	247.2 (18,17)	242.2 (17,16)	247.8 (17,16)
32	261.5 (20,19)	261.1 (19,18)	269.2 (19,18)	262.4 (18,17)
34	274.9 (21,20)	275.1 (20,19)	283.5 (20,19)	277.2 (19,18)

Table 7: $\log_2$ cost of a run, and the (d, s) that achieves it, as a function of m and t , with the four observations of Section 3 (the search that gives Table 2). The cost does not depend on n . The entries for $(m, t) = (12, 64)$, $(13, 96)$ and $(13, 128)$ are those of Table 2.

Table 8 gives examples. The first, $(m, n, t) = (18, 6176, 112)$, has a public key of 1.0 MB, the size of that of mceliece6688128, and ciphertexts of 252 bytes instead of 208. A run on it costs $2^{142.1}$. The last three rows keep the t of mceliece348864, mceliece460896 and mceliece6688128. They raise m until a run costs more than the claimed level of that set, 2^{143} , 2^{207} or 2^{272} . They then raise n until a Stern-type estimate of ISD is back at the value it has for that set. The public keys come to 0.59, 2.1 and 7.1 MB, which is 2.3, 4.0 and 6.8 times the present sizes. With memory charged as in Section 9, smaller m suffice. For example, $m = 15$ and $t = 64$ give 2^{120} bit operations, and 2^{150} with $M^{1/2}$.

These rows illustrate the scaling. They are not proposals. The run costs in them rest on Assumptions 1 and 2 far outside the range in which we tested them, which can only make the attack look cheaper than it is. We did not examine how other attacks behave at large m or at small $n/2^m$, and the ISD column is a rough estimate. What the rows show is that, in the cost model of Table 1, a larger m counters the attack at the price of a factor of two to seven in key size. The algebra does not rule out binary Goppa parameters with keys of half a megabyte to a few megabytes. For the *existing* parameter sets, an argument that they meet their claimed levels has to be an argument about the cost model (Section 9).

Acknowledgement. We thank Ashrujit Ghoshal, Yuval Ishai, Aayush Jain and Nuo Zhou Sun for sharing their key-recovery algorithm before publication, and for correspondence. Section 5 is built on the central observation of their algorithm. The improvements of Section 3 and the single-run reading of Section 5 are the suggestions recorded in their Remarks 5.1 and 7.3. We thank Daniel J. Bernstein for identifying an error in the first version’s discussion of key sizes, and Sanketh Menda for identifying errors in the cost figures of earlier versions.

AI Disclaimer. This work was generated by using Claude.

set	(m, n, t)	public key ciphertext			$\log_2$ cost of a run			ISD (Stern)
		(bytes)	(bytes)	(d, s)	$\log_2 N$	bit ops	$\times M^{1/2}$	
mceliece348864	(12, 3488, 64)	261 120	96	(7, 6)	38.4	94.1	118.0	151.4
mceliece460896	(13, 4608, 96)	524 160	156	(7, 6)	40.8	99.3	124.6	193.3
mceliece6688128	(13, 6688, 128)	1 044 992	208	(7, 6)	42.2	102.4	128.4	268.0
$m = 18$, key of 6688128	(18, 6176, 112)	1 048 320	252	(10, 9)	61.3	142.1	177.9	194.4
t and ISD of 348864	(18, 5248, 64)	589 824	144	(11, 10)	62.9	145.1	181.6	152.3
t and ISD of 460896	(26, 9216, 96)	2 096 640	312	(15, 14)	92.6	207.1	258.8	194.2
t and ISD of 6688128	(34, 17344, 128)	7 067 648	544	(20, 19)	129.2	283.5	353.9	268.6

Table 8: Examples with $n < 2^m$. Sizes are $mt(n - mt)/8$ and $mt/8$ bytes. The cost of a run is computed as for Table 7. The memory-charged column takes $M = 4N(k_\ell + 16)$ bits, as in Section 9. The last column is the $\log_2$ bit complexity of Stern’s algorithm as estimated by the CryptographicEstimators library [10], version 2.1.1. For the five Classic McEliece sets this estimate is 151.4, 193.3, 268.0, 268.4, 303.2, against the 151, 191, 257, 257, 287 of [7] used elsewhere in this paper. It serves only to compare rows with each other. In the last three rows n is the smallest multiple of 64 at which the estimate reaches that of the set named.

A The recovered key of TII-253

Public key: file `pk_McEliece_253.txt` of [17], with SHA-256

`a66ac1368bd6d311838e3a255c2d12766818731739c53d1ec79a46c8838ee793`.

Parameters: $m = 8$, $t = 9$, $n = 214$, $k = 142$, field $\mathbb{F}_{2^8} = \mathbb{F}_2[a]/(a^8 + a^4 + a^3 + a^2 + 1)$. Below, a field element is written as the byte whose bit i is the coefficient of a^i . The public code equals $\text{Goppa}(L, g)$ for the monic irreducible

$$g = x^9 + \mathbf{a6} x^8 + \mathbf{94} x^7 + \mathbf{4a} x^6 + \mathbf{36} x^5 + \mathbf{82} x^4 + \mathbf{72} x^3 + \mathbf{38} x^2 + \mathbf{d7} x + \mathbf{be}$$

and the support L (one byte per coordinate, in the column order of the public key, 32 per line):

```
2256e1ae20345840a00e4ede95fd1bf97fd0f3e970211367146f510169d9c890
f7c4f65ebb97b5db2a80386052baf4c2d12fd7746278cb9d025b23c679720dec
63cf9bc5b3b05a0766aa067d4c5d541e827be3a705753d4b4d88cef2fe177c29
a2b8a19a3e819c6ebd12fac75f761dbc4864100c4fe48bc341b2b41f683fa647
31d56de6aff10bf50a7133c0ac2cdf308659fca58796a33cc937d6ad8d49fbb1
4a3b65b7dceea4b639361ab918da0477dd4332cd83cc8f57ed2b7325e2f82755
94ef351528e798619e2442ea84a9c1eb2e998593260f
```

In the two-line text format of the challenge (coefficients of g from low to high degree, then L , as polynomials in a) this key is accepted by the organisers’ script `check_solution_track2.sage`.

References

- [1] M. R. Albrecht, D. J. Bernstein, T. Chou, C. Cid, J. Gilcher, T. Lange, V. Maram, I. von Maurich, R. Misoczki, R. Niederhagen, K. G. Paterson, E. Persichetti, C. Peters, P. Schwabe, N. Sendrier, J. Szefer, C. J. Tjhai, M. Tomlinson, and W. Wang. Classic McEliece: conservative code-based cryptography: cryptosystem specification. Round-4 submission to the NIST post-quantum standardization process, October 2022. <https://classic.mceliece.org>.
- [2] The Classic McEliece team. Classic McEliece: conservative code-based cryptography: guide for security reviewers, October 2022. <https://classic.mceliece.org>.

- [3] M. R. Albrecht and G. V. Bard. The M4RI library. <https://github.com/malb/m4ri>.
- [4] D. Apon. An algebraic-geometry lower bound against the ePrint:2026/1747 McEliece key-recovery attack. Cryptology ePrint Archive, Paper 2026/1810, 2026.
- [5] M. Bardet, A. Lemoine, and J.-P. Tillich. An attack on the CFS scheme and on TII McEliece challenges. Cryptology ePrint Archive, Paper 2026/430, 2026.
- [6] D. J. Bernstein. `break-fake-mceliece-sizes-from-tii.sh`. Script, 22 July 2026. <https://cr.yp.to/2026/20260722-break-fake-mceliece-sizes-from-tii.sh>.
- [7] D. J. Bernstein and T. Chou. CryptAttackTester: high-assurance attack analysis. In *CRYPTO 2024, Part VI*, volume 14925 of *LNCS*, pages 141–182. Springer, 2024.
- [8] P. Briaud, A. Lemoine, H. Randriambololona, and J.-P. Tillich. A heuristic subexponential attack on the McEliece cryptosystem. Cryptology ePrint Archive, Paper 2026/1232, 2026.
- [9] A. Couvreur, R. Mora, and J.-P. Tillich. A new approach based on quadratic forms to attack the McEliece cryptosystem. In *ASIACRYPT 2023, Part IV*, volume 14441 of *LNCS*, pages 3–38. Springer, 2023.
- [10] A. Esser, J. Verbel, F. Zweyding, and E. Bellini. CryptographicEstimators: a software library for cryptographic hardness estimation. Cryptology ePrint Archive, Paper 2023/589, 2023. Python package `cryptographic-estimators`.
- [11] J.-C. Faugère, V. Gauthier-Umaña, A. Otmani, L. Perret, and J.-P. Tillich. A distinguisher for high-rate McEliece cryptosystems. *IEEE Trans. Inform. Theory*, 59(10):6830–6844, 2013.
- [12] R. Fröberg. An inequality for Hilbert series of graded algebras. *Math. Scand.*, 56:117–144, 1985.
- [13] A. Ghoshal, Y. Ishai, A. Jain, and N. Sun. Quasipolynomial cryptanalysis of the McEliece cryptosystem (or: PIR meets McEliece). Cryptology ePrint Archive, Paper 2026/1630, 2026.
- [14] T. Hemmert. Key recovery for the McEliece cryptosystem using higher-order vanishing. Cryptology ePrint Archive, Paper 2026/1339, 2026.
- [15] T. Hemmert and A. Wiemers. Distinguishing Goppa codes using higher-order vanishing. In *Advances in Cryptology – CRYPTO 2026*, LNCS, pages 191–223. Springer, 2026.
- [16] E. Kirshanova and A. May. Breaking Goppa-based McEliece with hints. *Information and Computation*, 293:105045, 2023.
- [17] E. Kirshanova and A. Esser. TII McEliece challenges: scripts and instances. https://github.com/ElenaKirshanova/tii_decoding_challenge, 2023; challenge description at <https://crowdchallenge.tii.ae/mceliece-challenges/>.
- [18] R. J. McEliece. A public-key cryptosystem based on algebraic coding theory. DSN Progress Report 42–44, Jet Propulsion Laboratory, 1978.
- [19] P. L. Montgomery. A block Lanczos algorithm for finding dependencies over $\text{GF}(2)$. In *EUROCRYPT ’95*, volume 921 of *LNCS*, pages 106–120. Springer, 1995.

- [20] H. Randriambololona. The syzygy distinguisher. In *EUROCRYPT 2025, Part VI*, volume 15606 of *LNCS*, pages 324–354. Springer, 2025.
- [21] M.-J. O. Saarinen. HOVER: higher-order vanishing endomorphism recovery. Cryptology ePrint Archive, Paper 2026/1778, 2026. Recovered keys at <https://github.com/mjosaarinen/tii-solved>.
- [22] M.-J. O. Saarinen. Bit operation cost of hold-out key-recovery attacks against Classic McEliece. Cryptology ePrint Archive, Paper 2026/1786, 2026. Artifact at <https://github.com/mjosaarinen/mccost>.
- [23] M.-J. O. Saarinen. Two-anchor holdout/Hermite: solving the TII-254 McEliece key recovery challenge. Cryptology ePrint Archive, Paper 2026/1986, 2026.
- [24] The Sage Developers. SageMath, the Sage Mathematics Software System. <https://www.sagemath.org>.
- [25] V. M. Sidelnikov and S. O. Shestakov. On insecurity of cryptosystems based on generalized Reed–Solomon codes. *Discrete Math. Appl.*, 2(4):439–444, 1992.
- [26] K. Vedenev. Extending distinguishing to key recovery for subfield subcodes of GRS codes. Cryptology ePrint Archive, Paper 2026/1747, 2026. Working draft.
- [27] C. Wieschebrink. Cryptanalysis of the Niederreiter public key scheme based on GRS subcodes. In *PQCrypto 2010*, volume 6061 of *LNCS*, pages 61–72. Springer, 2010.